

SK1002 · AI 辅助应用开发 · 2026-2027 学年第一学期

AI 辅助应用开发基础 (二)

Vibe Coding 核心方法 —— 从提示词到协作闭环

授课教师：张贤和

工号：3036

本讲内容

01

Vibe Coding 核心方法

明确意图 · 提供上下文 · 小步实现 · 持续验证 · 及时纠偏

02

实验三 · 重制作品集

结构化提示词 · 局部修改 · 分轮迭代 · 运行验证

03

实验二 vs 实验三

同一任务 · 同一种工具 · 不同开发方式 · 结果可控性对比

本周要建立的核心认识：提示词是沟通手段 · Vibe Coding 是完整的人机协同过程

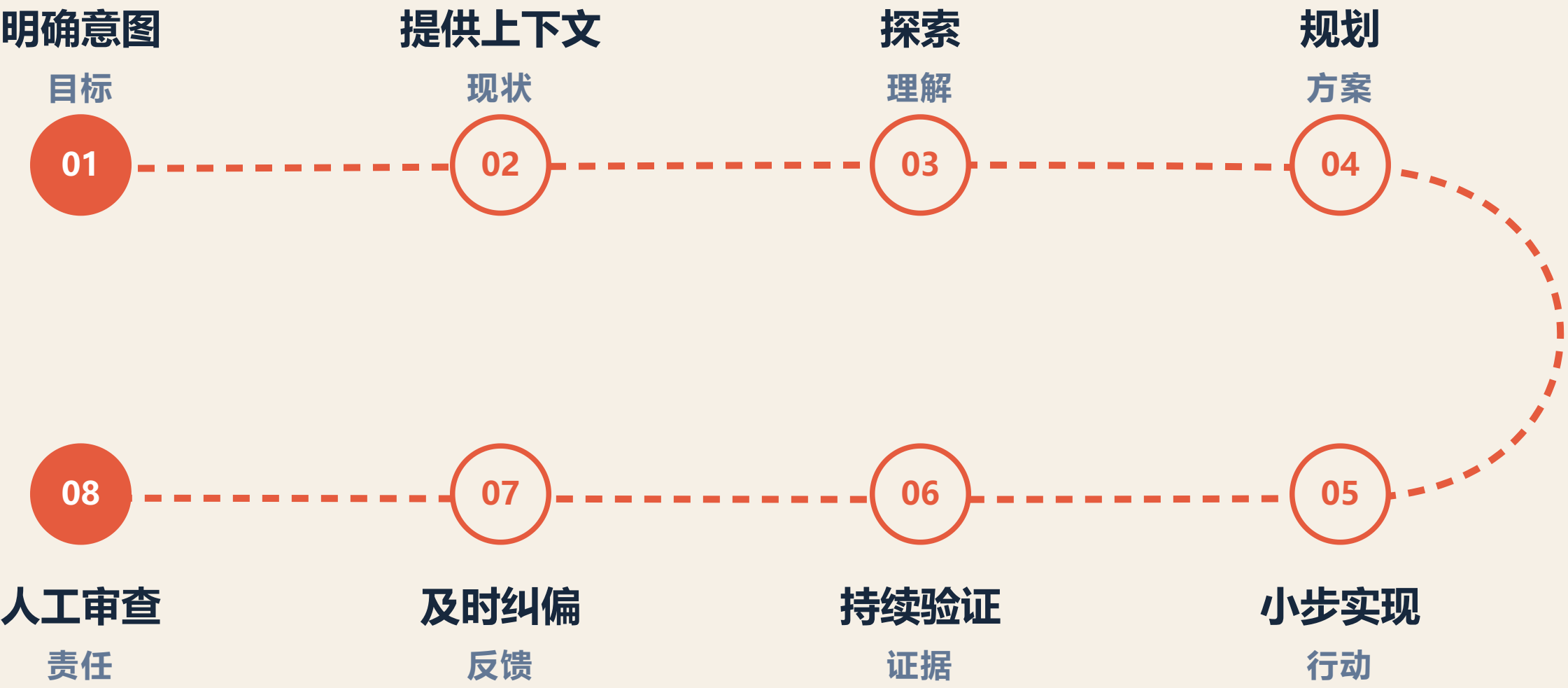
简单任务：三步闭环



适用：单个页面 · 小范围调整 · 结果一眼可判定

根据任务复杂度灵活使用，不为“走流程”而走流程。

复杂任务：把一次生成变成一条可控路径



任务越复杂，越需要可停、可查、可纠偏。

第一步不是写代码，而是说清“要什么”

模糊描述

“帮我做一个好看的
个人作品集网站。”

AI 还需要猜：

展示什么？ 有哪些功能？

能改哪里？ 技术限制是什么？

怎样才算完成？

结构化需求

目标 —— 创建个人作品集网站

功能 —— 个人介绍 · 项目 · 关于我 · 联系方式

约束 —— HTML / CSS / JS · 不使用 UI 库

验收 —— 运行并检查内容、交互与响应式

先定义“怎样才算完成”，再让 AI 开始实现。

结构化提示词：AI 需要知道什么？



不是把话说长，而是让 AI 少猜一点。

核心判断

好的提示词，不是越长越好 而是让 AI 少猜一点

无效长度

更多形容词

重复同一要求

空泛期待“高级感”

有效信息

明确目标

真实上下文

可判定的验收标准

信息密度 ↑ → 猜测空间 ↓ → 结果可控性 ↑

告诉 AI“现在是什么情况”



上下文不是越多越好，而是与当前任务相关。

一句话的差异，决定 AI 要猜多少

上下文不足

“把这个页面改成卡片式布局。”

AI 不知道

页面在哪里？

哪些内容不能动？

是否允许安装依赖？

需要复用什么？

上下文充分

请只修改个人作品集页面中的“项目作品”区域；保持已有内容和功能不变。三张卡片包含名称、时间、简介、技术栈和图片。

优先复用已有结构与样式，不要引入第三方 UI 组件库。

信息足以支持判断，AI 才能控制修改边界。

把大目标拆成可验证的小任务

理性使用 AI
让开发更高效
也更可控

Vibe Coding 不应追求“一次提示生成整个系统”

更有效的方法是拆分任务、逐步实现、持续验证

复杂的系统
来自清晰的步骤
而不是一次性的魔法



不推荐：一次提示生成整个系统



“请帮我开发一个完整的个人作品集网站，包括项目列表、项目卡片、类别筛选、个人介绍等功能，并对网站进行移动端适配。”

容易带来的问题



架构问题
难以发现



需求遗漏



错误
相互影响



修改范围
失控



问题
难以定位



更合理：拆分任务，逐步实现



大目标 → 小任务 → 逐步实现



每完成一个阶段，就进行一次检查和反馈。



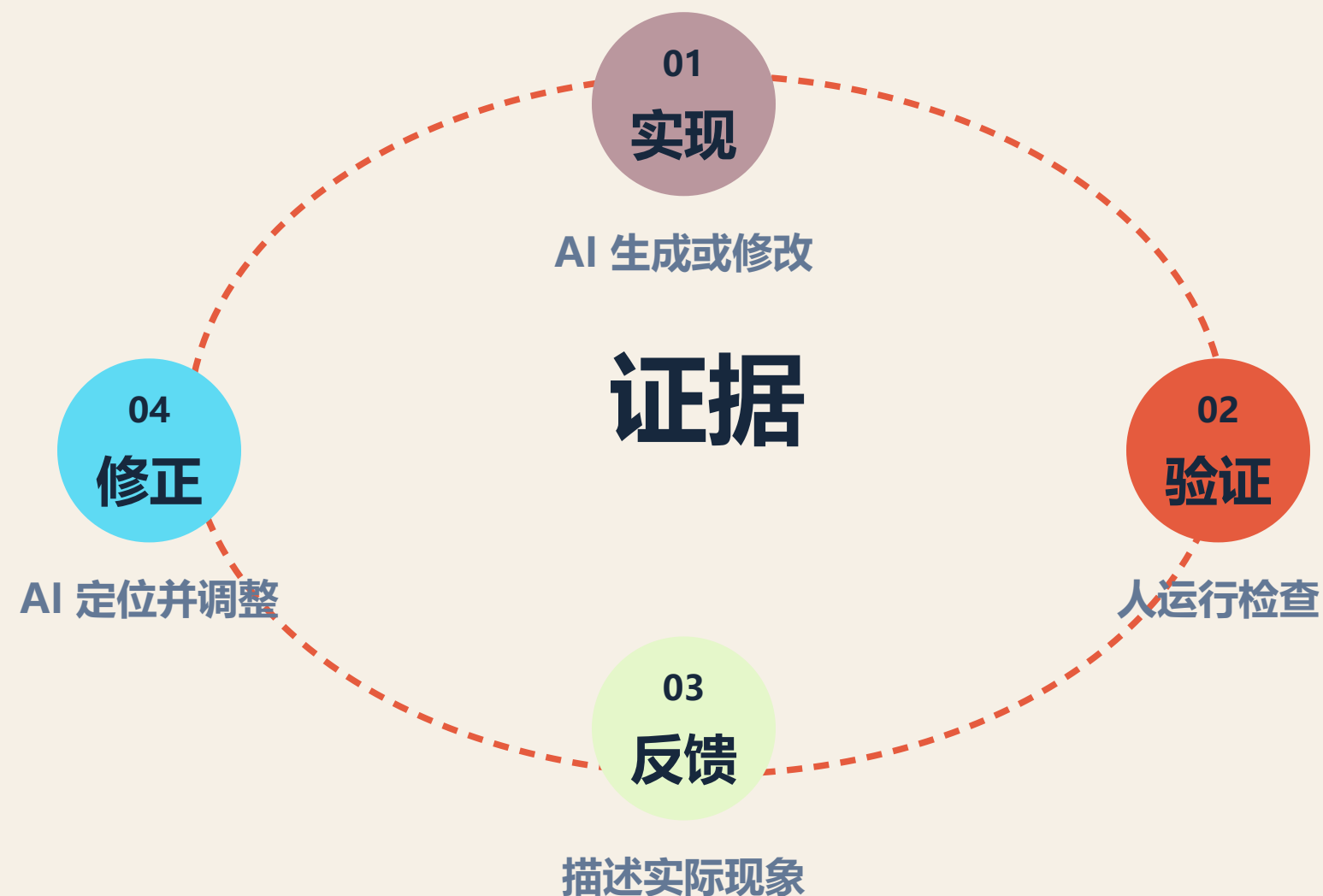
总结

一次性生成全部功能，往往**放大复杂度**；分阶段推进，才更容易发现问题、控制改动并保证质量。

更小的步骤
带来更可靠的结果



“AI 完成了” ≠ “功能正确”



AI 的回答是“主张”，运行结果才是“证据”。

看真实结果，而不是只看回复

01

浏览器运行

页面是否真正打开

02

点击交互

筛选是否按预期工作

03

页面显示

内容是否完整、层级清晰

04

响应式

桌面与手机尺寸是否正常

05

控制台

是否出现错误或警告

06

自动测试

关键行为能否重复验证

07

类型 / Lint

代码质量问题是否暴露

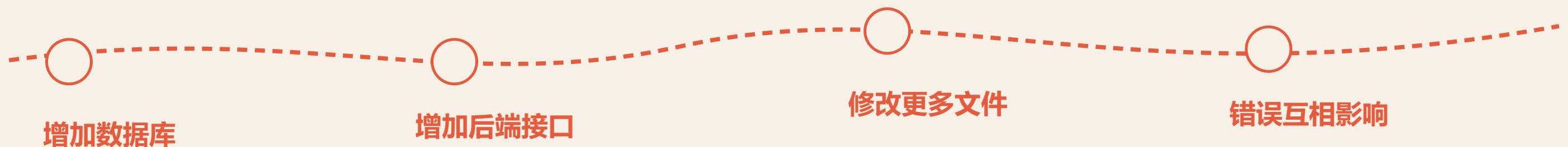
08

项目构建

交付版本能否成功生成

快速反馈，而不是快速堆代码

错误方向继续



有效纠偏表达



越早纠偏，越少代码需要返工。

复杂任务：先理解，再行动

01

探索

现在是什么情况？

读取目录与相关文件
识别技术栈、页面结构
定位组件与代码

02

规划

准备怎么做？

列出修改文件
说明每个文件职责
设计实现与验证方法

03

实现

按确认方案行动

控制修改范围
小步提交结果
每一步之后运行验证

探索、规划阶段明确写：先不要修改代码。

AI 负责实现，人负责判断

	AI-提高效率	人-对质量负责
需求	整理需求、发现歧义	明确真正需求
探索	阅读代码、分析结构	判断关键业务逻辑
方案	提供实现选择	选择技术路线
实现	生成与修改代码	控制范围与约束
验证	运行测试、分析错误	确定验收标准
交付	辅助部署流程	承担最终责任

AI 可以代替部分操作，但不能代替人的责任。

LAB 03 • 实验三

用结构化需求重新开发个人作品集网站

任务不变，只改变开发方式

03

先用完备提示词从零构建

01	目标	个人作品集
02	上下文	纯前端
03	范围	从零创建
04	约束	不使用 UI 库
05	验收	运行并检查
06	输出	HTML / CSS / JS 分离

PROMPT · 关键内容

请帮我开发一个个人作品集网站，用于展示个人介绍、技能方向和项目作品，面向老师、同学和企业招聘人员。

- 页面内容
- 视觉风格
- 技术要求
- 验收要求
- 输出要求

与实验二结果并排对比

实验二 · 简单提示词

实验三 · 完备提示词

区域与内容

AI 自主决定

项目作品 + 关于我

卡片信息

可能缺项

六项信息明确要求

文件组织

可能写在一个文件

HTML / CSS / JS 分离

响应式

未必处理

桌面端 + 移动端

差异来源

默认经验与猜测

提示词中的明确意图

圈出：哪些差异能直接追溯到某一条提示要求？

提供上下文，进行局部修改

SCOPE FENCE · 修改边界



允许修改

仅“项目作品”区域

三张独立卡片 · 补齐六项信息 · 提升信息层次



保持不变

“关于我”区域与已有功能

优先复用已有结构和样式



明确禁止

不扩大任务范围

不增加第三方依赖 · 不擅自重构其他区域

范围越清楚，越容易判断 AI 是否“越界”。

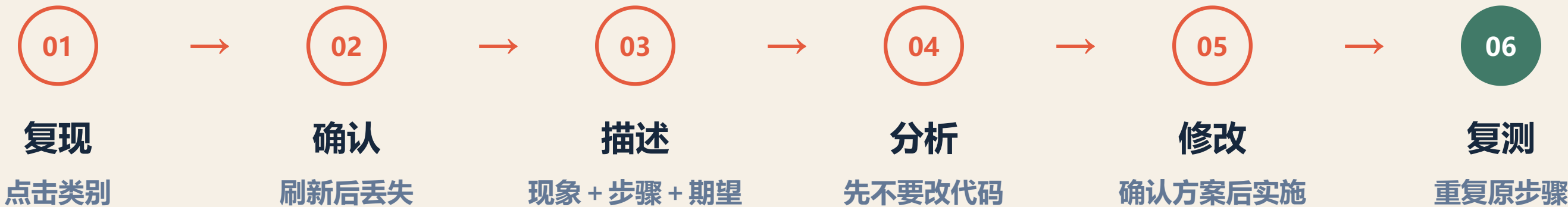
小步迭代：每轮只解决一类问题



每一轮：需求 → AI 实现 → 运行 → 反馈 → 再实现。

验证并修复：用证据驱动下一轮

发现问题 点击类别后筛选正常，但刷新浏览器后恢复为显示全部项目。



发现问题 → 描述问题 → AI 分析 → 修改 → 再验证。

开发方式改变了什么？

实验二 · 简单提示词

实验三 · 完备提示词

需求描述	模糊、简略	明确、完备
上下文	较少	与任务相关且充分
开发方式	一次生成	从零构建 + 小步迭代
AI 猜测	较多	较少
修改范围	容易扩大	明确控制
验证方式	主要看 AI 回答	实际运行验证
问题处理	重新描述	持续反馈纠偏
结果可控性	较低	较高

完备提示词减少偏差，但不能 100% 消除偏差。

用你的实验结果回答，而不是凭感觉

01 同一种 AI，为什么两个版本结果不同？

02 哪些提示信息对结果影响最大？

03 哪些上下文帮助 AI 更准确地完成任务？

04 为什么小步迭代更容易控制结果？

05 为什么“AI 完成了”不能证明功能正确？

每个判断至少引用一个可观察证据。

Vibe Coding: 建立可控的人机协同过程



人表达意图 → AI 实现 → 人验证 → 人反馈 → AI 迭代

NEXT WEEK

项目规则 · 版本管理 · 项目发布